

Respostas - Design Patterns

Integrantes: Raphaella Roma Mendes Alves e Roger Vinícius Pereira Augusto

1. Configuração da aplicação - Singleton

1.1 Por que usar `__new__` não impede novas execuções de `__init__`?

O método `__new__` é responsável por criar e retornar o objeto. Já o `__init__` inicializa o objeto retornado e pode ser executado novamente a cada chamada de `AppConfig()`, mesmo quando o objeto é o mesmo. Por isso, apenas controlar a criação em `__new__` não impede que um `__init__` redefina os atributos. Na nossa implementação, os valores iniciais são definidos dentro do `if` em `__new__`, somente quando a instância ainda não existe, e não usamos `__init__` nessa classe.

1.2 Como um módulo Python poderia compartilhar a configuração?

Poderíamos declarar `environment`, `currency` e `debug` diretamente em um módulo de configuração. Os outros arquivos importariam esse módulo e acessariam os valores por ele, como `config.currency`. Como o Python normalmente reutiliza o módulo já importado no mesmo processo, esses arquivos compartilhariam a configuração sem precisar de uma classe Singleton. Para observar alterações, os acessos seriam feitos pelo módulo, em vez de importar cada valor separadamente.

1.3 Qual uma possível consequência da configuração global compartilhada?

Uma alteração feita em uma parte do sistema afeta as outras partes que usam a mesma configuração. Por exemplo, mudar `debug` para `True` fica visível em todas as referências de `AppConfig`. Isso também exige cuidado nos testes, pois uma alteração pode interferir em outro teste se o valor não for restaurado.

2. Pedido e Builder

2.1 Quais componentes correspondem ao Builder e ao objeto construído?

O Builder é a classe `OrderBuilder`, que preenche o pedido pelos métodos `add_cliente`, `add_produto` e os métodos dos atributos opcionais. Esses métodos retornam `self`, permitindo encadear as chamadas. O objeto construído é uma instância de `Order`, que guarda os dados do pedido e calcula o total com `total()`. O método `build()` verifica se há um objeto `Client` como cliente, retorna o pedido e prepara o builder para uma nova construção.

2.2 Qual a diferença entre construir diretamente com Order e usar o Builder?

Na implementação atual, poderíamos chamar `Order()` e depois preencher seus atributos manualmente. Também seria possível adaptar o construtor para receber esses dados como parâmetros. O Builder organiza esse preenchimento em etapas e permite escolher os atributos opcionais com chamadas encadeadas. Além disso, `build()` rejeita um pedido sem cliente. Essa verificação está no Builder; criar `Order()` diretamente não faz essa validação.

3. Pagamento e Factory Method

3.1 Quais classes representam os papéis do padrão?

Creator: `PaymentProcessor`, que define `create_payment()` e contém o fluxo comum em `process_order()`. Concrete Creator: `PixProcessor`, `CreditCardProcessor` e `BoletoProcessor`, que implementam a criação de cada pagamento. Product: `Payment`, que define o método `pay(amount)`. Concrete Product: `PixPayment`, `CreditCardPayment` e `BoletoPayment`, que implementam o pagamento de cada forma.

3.2 Por que uma função com if/elif não caracteriza, por si só, Factory Method?

Uma função com if/elif pode escolher e criar objetos, mas isso sozinho não representa a estrutura do Factory Method. Na nossa implementação, a classe base define o fluxo de processamento e chama um método de criação que é implementado pelas subclasses. Assim, `process_order()` usa o pagamento retornado sem precisar instanciar diretamente PIX, cartão ou boleto.

3.3 O que precisaria mudar para adicionar outra forma de pagamento?

Na organização atual, adicionaríamos em `payment.py` uma nova classe que herda de `Payment`, com seu nome e sua implementação de `pay()`, e um novo processador que herda de `PaymentProcessor`, retornando esse pagamento em `create_payment()`. Também acrescentaríamos um exemplo ou teste e escolheríamos o novo processador onde a aplicação é iniciada. O fluxo de `process_order()` e as classes dos pagamentos existentes não precisariam ser alterados. O nome registrado no pedido deve corresponder ao nome do pagamento criado, pois o fluxo verifica essa correspondência.

4. Famílias por canal - Abstract Factory

4.1 Por que checkout e notificação formam uma família de produtos?

Checkout e notificação são objetos diferentes, mas estão relacionados ao mesmo canal de venda. No canal WEB, a família é formada por `WebCheckout` e `WebNotification`. No MOBILE, ela é formada por `MobileCheckout` e `MobileNotification`. Cada fábrica cria os dois objetos correspondentes ao seu canal.

4.2 Qual problema a Abstract Factory resolve nessa situação?

A Abstract Factory reúne a criação dos objetos de cada canal e evita que o código que os utiliza tenha que escolher as classes concretas separadamente. A função `process_channel()` recebe uma `ChannelFactory`, obtém o checkout e a notificação e chama `show()` e `send()`. Podemos passar `WebFactory` ou `MobileFactory` sem alterar essa função, mantendo os produtos da mesma família.

4.3 Por que o pagamento não deve fazer parte da fábrica do canal?

O canal de venda e a forma de pagamento são escolhas independentes. Um pedido WEB pode usar PIX, cartão ou boleto, e o mesmo vale para MOBILE. Se a fábrica do canal também escolhesse o pagamento, ela misturaria essas decisões. Por isso, as fábricas de canais criam apenas checkout e notificação, enquanto os processadores de pagamento ficam separados em `payment.py`.

5. Seleção de fábrica e alteração do sistema

5.1 Quais arquivos foram criados ou alterados para adicionar o KIOSK?

Para esta parte foi criado um único arquivo novo, o `registry.py`. É nele que ficam o registro de fábricas de canal, a função `get_channel_factory` e o registro inicial de WEB e MOBILE. A inclusão do KIOSK foi feita por acréscimo dentro desse mesmo arquivo, adicionamos as classes `KioskCheckout`, `KioskNotification` e `KioskFactory` e uma linha registrando a nova fábrica. A função `get_channel_factory` continuou exatamente igual, e nem nenhum outro arquivo. O código que usa as abstrações `Checkout` e `Notification` também não precisou saber que o KIOSK passou a existir. Para adicionar um canal, basta escrever as classes dele e registrar, sem editar a lógica de seleção.

5.2 As alterações são compatíveis com o princípio OCP?

Sim. O OCP diz que um componente deve estar fechado para modificação e aberto para extensão. A função de seleção está fechada porque ela não decide nada com base no nome do canal, ou seja, não existe uma série de ifs escolhendo fábricas. Ela apenas consulta um registro e devolve a fábrica correspondente. E o sistema está aberto para extensão porque novos canais entram por esse registro, sem alterar nada do que já existe. Como o KIOSK foi adicionado sem modificar o comportamento anterior, apenas somando um caso novo, a mudança respeita o OCP. Vale notar ainda que um canal desconhecido gera um erro informando inclusive os canais registrados, em vez de falhar silenciosamente devolvendo um valor nulo.

6. Responsabilidades e integração

6.1 Qual a responsabilidade principal de cada componente?

O EventLogger tem a função de registrar os eventos do fluxo, ele guarda as mensagens e permite consultar o histórico depois.

O OrderService tem a função de coordenar o fluxo, ele recebe o pedido pronto, a fábrica do canal e o processador de pagamento e apenas os organiza na ordem certa, que é apresentar o checkout, processar o pagamento e enviar a notificação.

O registry, com as funções de registrar e obter fábricas, tem a função de selecionar a fábrica de canal pelo nome.

As classes KioskFactory, KioskCheckout e KioskNotification representam a família de produtos do canal KIOSK.

Além desses componentes, temos os que já existiam: o OrderBuilder constrói o pedido, o método total() calcula o valor, o PaymentProcessor cria e executa o pagamento e a ChannelFactory cria a dupla de checkout e notificação de cada canal.

6.2 Três componentes e uma mudança que ficaria restrita a cada um.

No EventLogger, se quisermos passar a gravar os eventos em um arquivo ou em um banco de dados, em vez de mantê-los em memória, a mudança fica inteira dentro dele, e nenhum outro componente precisa saber como o log é armazenado. No KioskCheckout, se o texto ou o formato da tela de autoatendimento mudar, a alteração fica restrita a essa classe, sem afetar os canais WEB e MOBILE. No PayPalPayment, se a mensagem ou a regra específica do PayPal mudar, isso fica contido nele, sem impacto nas outras formas de pagamento nem no fluxo comum.

6.3 Uma decisão de projeto que poderia ser diferente.

No OrderService, optamos por receber o processador de pagamento como parâmetro do método que coloca o pedido. Assim, a forma de pagamento pode variar de um pedido para outro sem precisar de nada além disso. Uma alternativa seria injetar um único processador fixo no construtor do OrderService, o que deixaria a chamada um pouco mais curta, mas deixaria cada instância do serviço com uma só forma de pagamento, de modo que, para oferecer PIX e cartão no mesmo sistema, precisaríamos de várias instâncias ou de reconfigurar o serviço a cada pedido. Por isso preferimos passar por parâmetro.

7. Testes e alterações

Os três testes adicionais estão no arquivo testes_adicionais.py

O teste do Singleton verifica o compartilhamento de um atributo criado depois da inicialização. Ele cria em uma referência um atributo que não existia antes e confere que outra referência enxerga o mesmo valor. Isso é importante porque prova algo mais forte do que os exemplos do enunciado, que alteravam um atributo já existente. Mesmo um atributo criado depois é compartilhado, o que confirma que as duas referências são de fato o mesmo objeto, e não cópias, exatamente o que se espera de uma configuração global única.

O teste do Builder verifica o reinício após o build, ou seja, ele usa o mesmo builder para construir dois pedidos em sequência e confere que o segundo não herda os produtos do primeiro, então o primeiro fica com um produto e o segundo com nenhum. O resultado esperado é esse isolamento entre as duas construções. É importante porque o build chama o reset, e esse teste garante que um pedido não leve itens para o próximo, um erro que passaria despercebido nos exemplos originais.

O teste da fábrica verifica a coerência da família de produtos. Ele pega a fábrica do canal MOBILE pelo registro e confere que ela cria um checkout MOBILE e uma notificação MOBILE, ou seja, produtos da mesma variante. O resultado esperado é que ambos sejam do tipo MOBILE. Isso é importante porque é justamente o objetivo do Abstract Factory, que é garantir que a família de produtos criada por uma fábrica seja coerente entre si e não misture canais.

8. Situação de mudança - nova forma de pagamento

A forma de pagamento escolhida foi o PayPal e a implementação está no arquivo novo paypal.py.

8.1 Quais arquivos foram criados ou modificados?

Foi criado apenas o paypal.py, com as classes PayPalPayment, que é a forma de pagamento em si, e PayPalProcessor, que é o processador correspondente. O arquivo payment.py, onde ficam as formas de pagamento originais, não foi alterado. O service.py passa a instanciar o PayPalProcessor na demonstração da integração, mas isso é apenas uso do novo componente, e não uma modificação do mecanismo de pagamento.

8.2 O fluxo principal de processamento precisou ser alterado?

Não. O método process_order do PaymentProcessor continuou exatamente igual.

8.3 Quais classes existentes precisaram ser modificadas?

Nenhuma. As formas PIX, cartão e boleto e o próprio PaymentProcessor ficaram intactos.

8.4 Como o Factory Method contribuiu para essa extensão?

O process_order chama o método de criação de pagamento de forma polimórfica e trabalha sobre a abstração Payment, sem nunca depender de uma classe concreta. Para adicionar o PayPal, bastou criar uma subclasse do processador que sobrescreve esse método de criação devolvendo o novo tipo de pagamento. Como o fluxo comum depende só da abstração, ele passou a aceitar a nova forma de pagamento sem qualquer alteração, que é a aplicação do Factory Method.

8.5 Comparação com a inclusão do canal KIOSK.

As duas extensões se parecem no essencial, pois ambas foram feitas por acréscimo, respeitando o OCP, sem alterar o fluxo comum nem o código que consome as abstrações. Em ambos os

casos, o sistema trabalha com as interfaces, que são Payment de um lado e Checkout e Notification do outro, e não com as classes concretas. A diferença está no padrão e na forma de encaixe. O KIOSK usa Abstract Factory, pois acrescenta uma família de produtos que precisam ser coerentes entre si, que são o checkout e a notificação do canal, e essa família entra no sistema por um registro, sendo escolhida por um nome de canal na função de seleção. Já o PayPal usa Factory Method, pois acrescenta um único produto, que é a forma de pagamento, por meio de uma subclasse do processador, e é selecionado simplesmente instanciando esse processador, sem registro por nome. Em resumo, é o mesmo princípio de adicionar sem modificar, aplicado a granularidades diferentes: uma família coordenada em um caso e um único produto no outro.

Organização dos módulos

Os arquivos foram separados por responsabilidade, mantendo as classes relacionadas no mesmo módulo para deixar o projeto simples:

config.py: configuração compartilhada com AppConfig. order.py: cliente, produto, pedido e construção do pedido com OrderBuilder. payment.py: formas de pagamento e processadores que aplicam Factory Method. channels.py: checkouts, notificações e fábricas dos canais WEB e MOBILE. registry.py: registro e seleção de fábricas de canal e inclusão do canal KIOSK. service.py: coordenação do fluxo com o OrderService e registro de eventos com o EventLogger, além da execução completa da integração. paypal.py: nova forma de pagamento acrescentada na parte de extensão. testes_adicionais.py: os três testes adicionais solicitados.